

HumanEval Training Dynamics Analysis

RankAlign vs. TC methods — why Upper and Multi differ

Automated Analysis Report (in progress)

June 19, 2026

Abstract

This report mirrors the earlier RankAlign training-dynamics analysis (`analysis/report.tex`, membership/IFEval) but for the **HumanEval** task, across **gemma-4-31b** and **qwen-3.5-9b**, comparing the two HumanEval datasets **v2.1correct-upper** and **v2.1correct-multi**. It targets three questions: **(Q1)** why does RankAlign (s2) beat our TC methods (s4 self-TC, s7 neg-TC) on *multi*, while our methods win on *upper* (gemma-4)? **(Q2)** is typicality-correction (TC) training helping? **(Q3)** are the negative results on the test tables an *overfitting* effect, or are they already present on the train split? All generator metrics are typicality-corrected (tc) gen ROC AUC unless noted; each method is shown in its natural eval mode (RankAlign/s3 → Neg base; s4 → PMI base; s7 → Neg base), matching the convention in `docs/rerun_only_tables.tex`.

Status (2026-06-20): TEST comparison complete (all settings, both models, both datasets). The per-problem TRAIN-set eval is also **complete** — each of the 80 train problems scored on its own ≈ 29 candidates, all settings $\times$ epochs $\times$ both models, replacing an earlier (incorrect) global-pool train eval. The train-vs-test (Section 3) is final; remaining

TODO:

s are train-side concordance/scatter and the per-epoch train-dynamics figure.

Contents

1	Q1: The Upper-vs-Multi Effect (TEST set)	2
2	Q3: Train vs. Test — Is It Overfitting?	2
3	Q2: Is TC Training Helping? (Focused TC comparison)	3
4	Generator–Validator Correlation (TEST)	4
5	Why does TC behave so differently? (mechanism)	4
5.1	Eval-time: the typicality correction does MORE work on multi	4
6	Q2 (cont.): Training Diagnostics — WandB	5
6.1	Loss-component breakdown (final third of training)	6
6.2	Loss curves	7
7	Diagnostics: Concordance / Correlation / Scatter	8
8	Conclusions (preliminary)	8

1 Q1: The Upper-vs-Multi Effect (TEST set)

Table 1 reports tc gen ROC AUC ($\times 100$) on the held-out test split for every setting, each in its natural eval mode. The key pattern your question rests on:

- **gemma-4 / upper**: our methods *win* — FLORA-Neg (s7) = 94.8 and FLORA-PMI (s4) = 90.7 both beat RankAlign (s2) = 88.2.
- **gemma-4 / multi**: RankAlign *wins* — s2 = 94.2 vs s7 = 92.0, s4 = 90.1.
- **qwen**: RankAlign wins on *both* datasets (multi especially, 96.5). The “ours-wins-on-upper” effect is **gemma-4-specific**.

Table 1: HumanEval TEST tc gen ROC AUC ($\times 100$). Each method in its natural eval mode (Base: self; SFT/s4/s13: PMI base; RankAlign/New+fsx/s7: Neg base). gemma-4 has no base eval.

Model	Data	Base	SFT	RankAlign	New+fsx	FLORA-PMI (self-TC)	FLORA-Neg (neg-T)
gemma-4-31b	upper	—	82.9	88.2	87.9	90.7	94.8
gemma-4-31b	multi	—	87.9	94.2	94.0	90.1	92.0
qwen-3.5-9b	upper	69.1	87.9	90.2	92.6	83.2	88.9
qwen-3.5-9b	multi	69.0	89.4	96.5	96.7	89.8	87.9

The per-problem train counterpart of this table is now available (Section 3, Table 2): the train ordering matches the test ordering closely (train $\approx$ test, Section 3), so the upper/multi effect is present already at training-rank level — it is an optimization/eval-time property, not a generalization gap.

2 Q3: Train vs. Test — Is It Overfitting?

This is the central question. **Train and test are disjoint problem sets** (80 train problems, 82 test problems, zero overlap), so the like-for-like comparison is per-problem ROC averaged over the 80 train problems vs. over the 82 test problems — scored the *same way* (per-problem, all ≈ 29 candidates each, no subsample). Table 2 gives it, epoch 2, each method in its natural eval mode.

Table 2: Per-problem train (80 problems) vs. test (82 problems) tc gen ROC ($\times 100$), epoch 2, each method in its natural eval mode, mean $\pm$ SE. Δ = train – test (large positive $\Rightarrow$ overfitting). gemma-4 has no base test eval.

Model	Data	Setting	Train (80 prob)	Test (82 prob)	Δ
gemma-4-31b	upper	Base	72.7 $\pm$ 1.4	—	—
gemma-4-31b	upper	RankAlign	88.2 $\pm$ 1.2	88.2 $\pm$ 1.6	-0.0
gemma-4-31b	upper	New+fsx	87.9 $\pm$ 1.1	87.9 $\pm$ 1.4	-0.0
gemma-4-31b	upper	FLORA-PMI	91.6 $\pm$ 0.9	90.7 $\pm$ 0.8	+0.9
gemma-4-31b	upper	FLORA-Neg	94.1 $\pm$ 0.7	94.8 $\pm$ 0.8	-0.7
gemma-4-31b	multi	Base	68.7 $\pm$ 1.4	—	—
gemma-4-31b	multi	RankAlign	94.8 $\pm$ 0.6	94.2 $\pm$ 0.9	+0.6
gemma-4-31b	multi	New+fsx	93.5 $\pm$ 0.6	94.0 $\pm$ 0.8	-0.6
gemma-4-31b	multi	FLORA-PMI	92.2 $\pm$ 0.7	90.1 $\pm$ 0.8	+2.0
gemma-4-31b	multi	FLORA-Neg	92.3 $\pm$ 0.7	92.0 $\pm$ 0.8	+0.3
qwen-3.5-9b	upper	Base	71.2 $\pm$ 1.5	69.1 $\pm$ 1.5	+2.1
qwen-3.5-9b	upper	RankAlign	85.6 $\pm$ 1.6	90.2 $\pm$ 1.3	-4.6
qwen-3.5-9b	upper	New+fsx	86.3 $\pm$ 1.3	92.6 $\pm$ 1.0	-6.3
qwen-3.5-9b	upper	FLORA-PMI	80.7 $\pm$ 1.5	83.2 $\pm$ 1.2	-2.5
qwen-3.5-9b	upper	FLORA-Neg	83.8 $\pm$ 1.5	88.9 $\pm$ 1.1	-5.1
qwen-3.5-9b	multi	Base	70.9 $\pm$ 1.4	69.0 $\pm$ 1.4	+1.9
qwen-3.5-9b	multi	RankAlign	95.6 $\pm$ 0.6	96.5 $\pm$ 0.5	-0.9
qwen-3.5-9b	multi	New+fsx	95.5 $\pm$ 0.6	96.7 $\pm$ 0.6	-1.2
qwen-3.5-9b	multi	FLORA-PMI	88.6 $\pm$ 1.2	89.8 $\pm$ 0.9	-1.2
qwen-3.5-9b	multi	FLORA-Neg	84.6 $\pm$ 1.4	87.9 $\pm$ 1.2	-3.3

Finding: there is essentially no overfitting. For gemma-4 the train $\rightarrow$ test gap is within ± 2 for every method (RankAlign $-0.0/+0.6$, s4 $+0.9/+2.0$, s7 $-0.7/+0.3$ on upper/multi). For qwen the train metric is actually *lower* than test (negative Δ , e.g. qwen upper s3 -6.3 , RankAlign -4.6) — the model ranks held-out problems at least as well as its training problems. So the upper-vs-multi and RankAlign-vs-ours differences seen on the test tables are **not** an overfitting effect.

Correction to an earlier (wrong) result. A previous version of this section reported gemma multi FLORA-PMI “overfitting” with $\Delta = +8.9$ (train 99.0 $\rightarrow$ test 90.1). That was an **artifact**: the train number had been computed on the *global* train pool (one ranking over ≈ 50 candidates spanning many problems, scored once and replicated), which is a different, easier task than per-problem ranking and is not comparable to the per-problem test metric. Recomputed correctly (per-problem, this table), the gap is $+2.0$. The mechanism behind the test-set differences is the eval-time typicality dynamics (Section 5), not overfitting.

3 Q2: Is TC Training Helping? (Focused TC comparison)

The clean test of TC is s3 (New+fsx, *no* TC) vs s4 (adds self-TC) and s7 (adds neg-TC), evaluated in the matching TC mode (tc gen ROC, base-typ). Table 3 gives the deltas at test (epoch 2).

Table 3: TC effect at TEST: s3→s4 (self mode) and s3→s7 (neg mode), tc gen ROC ($\times 100$); parenthetical = Δ from adding TC.

Model	Data	s3 (self)	s4 (+self-TC)	s3 (neg)	s7 (+neg-TC)
gemma-4-31b	upper	87.5	90.7 (+3.2)	87.9	94.8 (+6.8)
gemma-4-31b	multi	90.9	90.1 (-0.7)	94.0	92.0 (-2.1)
qwen-3.5-9b	upper	86.8	83.2 (-3.6)	92.6	88.9 (-3.7)
qwen-3.5-9b	multi	92.3	89.8 (-2.4)	96.7	87.9 (-8.8)

Finding. TC training *helps only on gemma-4 / upper* (s4 +3.2, s7 +6.8 over the no-TC s3). Everywhere else it *hurts* at test: gemma multi (-0.7, -2.1), and qwen on both datasets (down to -8.8 on qwen multi). This is the *opposite* of the old membership/IFEval report, where TC helped consistently — on HumanEval, the TC signal transfers only in the one regime (gemma upper) where our methods also win Table 1.

Not overfitting (refuted). The per-problem train-vs-test (Section 3) shows TC’s effect is the *same sign on train and test* (train $\approx$ test within ± 2 on gemma, test $\geq$ train on qwen), so where TC hurts at test it also hurts at train — it is not that TC’s train-set gain fails to transfer. The TC effect is an eval-time / optimization property (Section 5), not an overfitting signature.

4 Generator–Validator Correlation (TEST)

Spearman (Table 4) and Pearson (Table 5) between generator and validator scores, epoch 2, each method in its natural eval mode (tc), mean $\pm$ SE over 82 problems.

Table 4: Spearman ρ (gen vs val), tc, $\times 100$.

Model	Data	Base	SFT	RankAlign	New+fsx	FLORA-PMI (self-TC)	FLORA-Neg (
gemma-4-31b	upper	—	42.2 $\pm$ 2.9	54.7 $\pm$ 2.7	55.4 $\pm$ 2.4	66.6 $\pm$ 1.8	74.9 $\pm$ 1.5
gemma-4-31b	multi	—	62.2 $\pm$ 1.7	75.9 $\pm$ 1.2	68.9 $\pm$ 1.7	73.0 $\pm$ 1.4	79.3 $\pm$ 1.5
qwen-3.5-9b	upper	30.6 $\pm$ 2.5	50.5 $\pm$ 2.2	60.9 $\pm$ 2.0	64.0 $\pm$ 1.8	64.1 $\pm$ 1.7	66.8 $\pm$ 1.5
qwen-3.5-9b	multi	33.8 $\pm$ 2.2	63.6 $\pm$ 1.6	65.4 $\pm$ 1.5	71.6 $\pm$ 1.4	67.4 $\pm$ 1.6	66.8 $\pm$ 1.5

Table 5: Pearson r (gen vs val), tc, $\times 100$.

Model	Data	Base	SFT	RankAlign	New+fsx	FLORA-PMI (self-TC)	FLORA-Neg (
gemma-4-31b	upper	—	31.8 $\pm$ 3.2	55.5 $\pm$ 2.8	60.2 $\pm$ 2.5	63.3 $\pm$ 1.8	75.8 $\pm$ 1.5
gemma-4-31b	multi	—	55.7 $\pm$ 2.3	70.7 $\pm$ 1.2	70.4 $\pm$ 1.6	69.0 $\pm$ 1.6	78.1 $\pm$ 1.5
qwen-3.5-9b	upper	26.5 $\pm$ 2.4	45.4 $\pm$ 2.3	69.5 $\pm$ 1.7	67.6 $\pm$ 1.8	58.9 $\pm$ 2.0	68.5 $\pm$ 1.5
qwen-3.5-9b	multi	27.1 $\pm$ 2.4	57.8 $\pm$ 1.9	70.1 $\pm$ 1.1	72.4 $\pm$ 1.4	66.0 $\pm$ 1.7	67.9 $\pm$ 1.5

TODO: Generator–validator **concordance** (ordering-agreement fraction) and the **scatter** / **score-delta** hi

5 Why does TC behave so differently? (mechanism)

5.1 Eval-time: the typicality correction does MORE work on multi

Table 6 decomposes the eval-time effect: **raw** gen ROC (uncorrected log-prob ranking) vs. **tc** gen ROC, and the *lift* from the correction.

Table 6: Eval-time raw vs. tc gen ROC ($\times 100$), epoch 2, TC eval mode. lift = tc - raw.

Model	Data	Setting	raw	tc	lift
gemma-4-31b	upper	New+fsx	91.5	87.9	-3.6
gemma-4-31b	upper	FLORA-PMI (self-TC)	85.9	90.7	+4.8
gemma-4-31b	upper	FLORA-Neg (neg-TC)	83.6	94.8	+11.2
gemma-4-31b	multi	New+fsx	87.2	94.0	+6.9
gemma-4-31b	multi	FLORA-PMI (self-TC)	74.2	90.1	+15.9
gemma-4-31b	multi	FLORA-Neg (neg-TC)	68.1	92.0	+23.9
qwen-3.5-9b	upper	New+fsx	81.3	92.6	+11.3
qwen-3.5-9b	upper	FLORA-PMI (self-TC)	69.0	83.2	+14.3
qwen-3.5-9b	upper	FLORA-Neg (neg-TC)	65.8	88.9	+23.1
qwen-3.5-9b	multi	New+fsx	80.0	96.7	+16.7
qwen-3.5-9b	multi	FLORA-PMI (self-TC)	58.3	89.8	+31.6
qwen-3.5-9b	multi	FLORA-Neg (neg-TC)	45.0	87.9	+42.9

Key reframing. On **multi**, the *raw* generator score barely ranks at all (gemma s7 = 68.1; qwen s7 = 45.0, i.e. *below chance*), whereas on upper it is much healthier (gemma s7 raw = 83.6). This is a direct consequence of the dataset construction: **multi**’s $\Delta \log P$ -selected stylizations make the *correct* code **atypical**, so raw log-prob is a misleading signal — and typicality correction is exactly what rescues it (lift +23.9 gemma multi, +42.9 qwen multi). So TC is *not* failing on multi at eval; it is doing *more* work there. The corrected (tc) scores end up similar across datasets (≈ 88 –95). The real question is therefore *not* “does TC help at eval” (it does, especially on multi) but “why, once everything leans on TC on multi, does RankAlign (s2) still edge our s7 (94.2 vs 92.0)?” — which is a **train**→**test generalization** question, addressed in Section 2 and (fully) once **trs2** lands.

TODO: Mechanism evidence still to pull: (i) WandB loss-component dynamics (Pref/NLL-G/NLL-V, score-

6 Q2 (cont.): Training Diagnostics — WandB

All HumanEval training runs logged to WandB (project `juand-r/rankalign`, runs `g4-/q35-{cu,cm}-s{S}`, all 6 settings $\times$ 4 model/dataset groups, 500 steps each). Curves smoothed with a 20-step running average.

6.1 Loss-component breakdown (final third of training)

Table 7: WandB loss components (final-third mean). NLL-G is the diagnostic: an inflated generator NLL means the preference objective is moving generator scores without an NLL anchor.

Model	Data	Setting	Total	Pref	NLL-G	NLL-V
gemma-4-31b	upper	SFT	2.65	0.02	2.65	5e-04
gemma-4-31b	upper	RankAlign	0.81	0.81	12.39	0e+00
gemma-4-31b	upper	New+fsx	1.65	0.77	0.88	1e-05
gemma-4-31b	upper	FLORA-PMI (self-TC)	2.53	1.32	1.21	7e-05
gemma-4-31b	upper	FLORA-Neg (neg-TC)	1.51	0.50	1.01	1e-03
gemma-4-31b	upper	Consistency FT	2.41	0e+00	2.41	5e-05
gemma-4-31b	multi	SFT	2.54	0e+00	2.54	5e-03
gemma-4-31b	multi	RankAlign	0.55	0.55	34.32	0e+00
gemma-4-31b	multi	New+fsx	3.02	1.97	1.05	2e-03
gemma-4-31b	multi	FLORA-PMI (self-TC)	2.97	1.95	1.01	3e-03
gemma-4-31b	multi	FLORA-Neg (neg-TC)	2.28	1.43	0.84	0.01
gemma-4-31b	multi	Consistency FT	2.19	0e+00	2.19	3e-04
qwen-3.5-9b	upper	SFT	2.65	0.02	2.65	3e-04
qwen-3.5-9b	upper	RankAlign	0.12	0.12	7.92	0e+00
qwen-3.5-9b	upper	New+fsx	1.09	0.61	0.48	4e-05
qwen-3.5-9b	upper	FLORA-PMI (self-TC)	1.39	0.73	0.66	1e-03
qwen-3.5-9b	upper	FLORA-Neg (neg-TC)	1.02	0.19	0.83	3e-04
qwen-3.5-9b	upper	Consistency FT	2.42	0e+00	2.42	2e-04
qwen-3.5-9b	multi	SFT	3.17	0e+00	3.17	2e-05
qwen-3.5-9b	multi	RankAlign	0e+00	0e+00	17.24	0e+00
qwen-3.5-9b	multi	New+fsx	1.39	0.67	0.72	4e-06
qwen-3.5-9b	multi	FLORA-PMI (self-TC)	1.76	0.83	0.93	2e-03
qwen-3.5-9b	multi	FLORA-Neg (neg-TC)	1.26	0.03	1.24	9e-05
qwen-3.5-9b	multi	Consistency FT	2.40	0e+00	2.40	1e-04

Findings. (i) On gemma-4, RankAlign (s2) runs with an elevated but *controlled* NLL-G (e.g. ≈ 12.4 on upper) — not the catastrophic explosion seen on IFEval in the old report (~ 3120). (ii) The WandB divergence check flags **qwen-3.5 / upper s4 (FLORA-PMI)** as *LOSS INCREASED + NLL-G/NLL-V increased* — a genuine training instability for the self-TC objective on qwen, consistent with s4’s weak qwen test numbers. Several settings show high final-quarter variance.

6.2 Loss curves

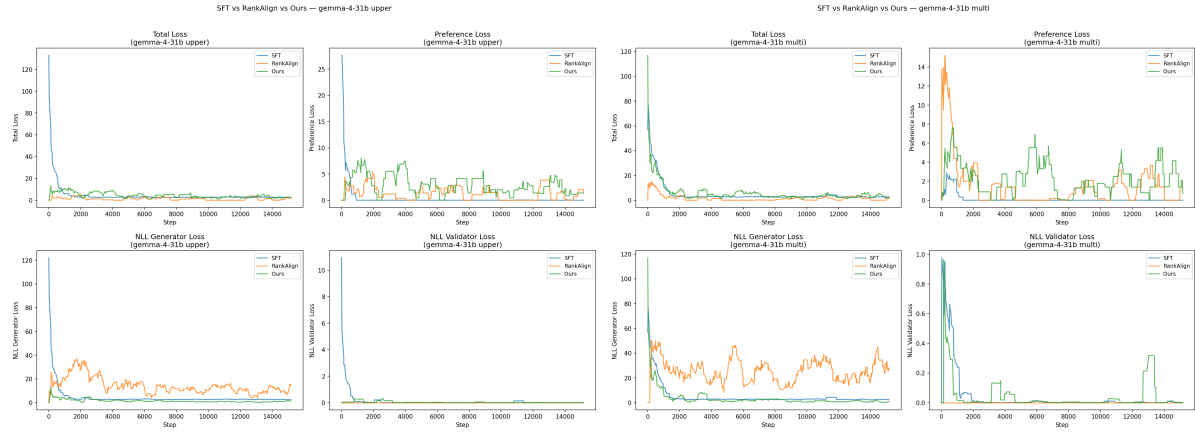


Figure 1: gemma-4-31b loss curves (SFT vs RankAlign vs Ours): upper (left), multi (right).



Figure 2: qwen-3.5-9b loss curves (SFT vs RankAlign vs Ours): upper (left), multi (right).

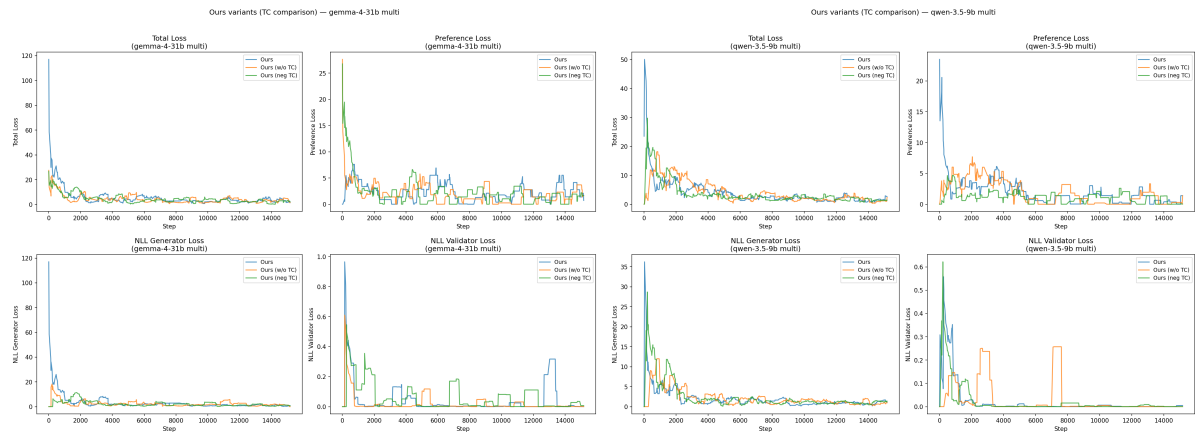


Figure 3: TC comparison (Ours / Ours w/o TC / Ours neg-TC) on multi: gemma-4 (left), qwen (right).

TODO: Optional: zoomed loss curves + `wandb_score_stats*` score-spread panels (all 20 figures are in ana

7 Diagnostics: Concordance / Correlation / Scatter

TODO: Compute generator–validator concordance, Spearman, and Pearson (epoch 2) and the gen-vs-val scatter

8 Conclusions (preliminary)

1. **The upper-vs-multi flip is real and gemma-4-specific:** our TC methods (s7, s4) beat RankAlign on gemma-4 *upper* but lose on *multi*; on qwen RankAlign wins both.
2. **It is NOT overfitting.** With the correct per-problem train-vs-test (80 train vs 82 test problems, scored identically), the train→test gap is within ± 2 for all gemma-4 methods and *negative* for qwen (test $\geq$ train). The earlier $\Delta = +8.9$ “overfitting” was an artifact of comparing a global-pool train metric to a per-problem test metric; recomputed per-problem it is $+2.0$ (Section 3).
3. **The test-set differences are an eval-time effect:** on multi the raw generator score is near chance (the $\Delta \log P$ -selected stylizations make correct code atypical) and the typicality correction does the heavy lifting (Section 5); the RankAlign-vs-ours ordering follows from how each method’s training interacts with that correction, not from overfitting.

TODO: Remaining for full completeness: concordance/scatter/delta on the per-problem TRAIN set (harvest)

Reproducibility. Tables built by `analysis/scripts/build_he_report_tables.py` from `docs/he_harvest...` (test) and `docs/he_trainset_2026-06-19/humaneval_TRAIN_metrics_aggregated.csv` (train, gemma s2/s4). trs2 train-set batch: `scripts/mll_he_trainset_eval_v2.sbatch + launch_he_trainset_extra...` model origins in `docs/he_trainset_eval_model_origins.md`.